Student Attendance Management System Specification

Student record management, role-based access and daily attendance tracking

Version 1.2 | 29 October 2026 | Updated for dashboard and account provisioning | Revised draft for implementation

Purpose and intended outcome

Build a web application that allows teachers to maintain student records and record daily attendance, while students can securely log in to view only their own attendance history and summary. This document defines observable behavior and verification criteria for an initial release suitable for AI-assisted implementation, planning and testing.

Source requirement

The project brief selects React JS for the UI, FastAPI for the API and PostgreSQL for persistence. This revision adds two roles explicitly requested for the system: Student (read-only access to personal attendance) and Teacher (administrative access to student records and attendance). Detailed field rules and workflows are proposed implementation requirements.

How to read this specification

Requirements marked S come directly from the selected project and role decisions. Requirements marked P are proposed elaborations to make the application implementable and testable. “Shall” describes required behavior. Requirement IDs connect implementation, acceptance criteria and tests. AI is the development aid, not a required feature of the application.

Document map

Section	Coverage
1–2	Scope, assumptions, actors and permissions
3–4	Functional requirements, workflows and business rules
5–7	UI, data model and REST API
8–9	Errors, quality requirements and acceptance tests
10	Delivery sequence and change control

1 Scope and baseline assumptions

Included in the initial release

The system serves one school/class cohort for a simple demonstration. Students log in and can view only their own attendance summary and history. Teachers log in and can manage student records, mark and edit daily attendance, and view reports for all students. The frontend uses React JS, the backend uses FastAPI and PostgreSQL stores application data.

ID	Basis	Requirement
S01	S	The UI shall use React JS, the API shall use FastAPI, and PostgreSQL shall be the database.
S02	S	The system shall provide Student and Teacher roles.
S03	S	Students shall be able to view their own attendance and summary.
S04	S	Teachers shall be able to manage student records and attendance.
P01	P	Authentication shall be required for all student and teacher application pages and protected API routes.
P02	P	A student's identity shall be linked to exactly one student record.
P03	P	The initial release supports Present and Absent only; no late, excused, timetable or holiday workflow.
P04	P	One teacher role is sufficient; there is no separate super-admin role.
P05	P	The teacher account is provisioned for the demo; public self-registration is not available. Student accounts are created/provisioned by the teacher or seeded for the demo.

Excluded from this release

Parent accounts, multiple schools, multiple permission tiers, timetable and holiday management, Late/Excused statuses, biometric or QR attendance, email/SMS notifications, exports, file uploads and AI-based predictions are excluded.

2 Actors and access control

Capability	Student	Teacher
Log in and log out	Yes	Yes
View own attendance history and percentage	Yes	Yes
View other students' attendance	No	Yes
Create/edit/deactivate student records	No	Yes
Mark attendance for a date	No	Yes
Edit saved attendance	No	Yes
View class-wide attendance reports	No	Yes
Manage roles or create teacher accounts	No	No; outside MVP

Authorization rules

AC01 — Every protected page and API operation requires a valid authenticated session/token. Unauthenticated requests return HTTP 401.

AC02 — The server derives user ID and role from validated authentication context. Client-supplied role or student_id must never grant access.

AC03 — A Student may retrieve attendance only for the student record linked to their authenticated account. Attempts to request another student's records or call teacher-only endpoints return HTTP 403 or a consistent 404 for inaccessible resources.

AC04 — Only a Teacher may create/update/deactivate student records, mark attendance, edit attendance, or retrieve class-wide reports. FastAPI enforces these permissions even if a client calls endpoints directly.

AC05 — Passwords, if locally stored, shall be hashed using a suitable password-hashing library and never stored or logged in plain text. Secrets shall be kept out of source control.

Primary user journeys

Student: log in → open My Attendance → view attendance percentage and present/absent totals → filter or inspect history by date → log out.

Teacher: log in → manage students → open Mark Attendance → select date → mark each active student Present or Absent → save → view/edit daily register → review reports → log out.

3 Functional requirements

Authentication and accounts

FR01 [P] — The system shall provide a login form for both roles using the agreed credentials mechanism. Invalid credentials shall show a generic error and shall not disclose whether a username exists.

FR02 [P] — After successful login, the application shall route users to the correct role-specific landing page. The server, not the browser route, determines permissions.

FR03 [P] — The system shall provide logout and invalidate/remove the active client session/token. Protected API requests after logout shall fail authentication.

Student management — Teacher only

FR04 [S04] — A Teacher shall be able to create a student with name, unique register number and class/section. The system shall create a linked Student login account and present the initial credentials to the Teacher before they are assigned to the student.

FR05 [P] — A Teacher shall be able to edit student name, register number and class/section, search by name/register number, and deactivate a student.

FR06 [P] — Deactivation shall preserve historical attendance and prevent the student appearing in new attendance registers by default. Hard deletion is not supported.

Attendance — Teacher only

FR07 [S04] — A Teacher shall select a date and see the active student roster with each student's register number, name and status for that date.

FR08 [P] — For a date with no saved register, statuses shall initially be unmarked. The Teacher must explicitly select Present or Absent for every active student before saving.

FR09 [P] — Saving a complete register shall create one attendance record per active student for that date in one database transaction.

FR10 [P] — If attendance already exists for a date, the UI shall load saved statuses. Saving changes updates existing records and shall not create duplicates.

FR11 [P] — The system shall prevent more than one record for a student on the same date.

Student attendance view and reports

FR12 [S03] — A Student shall see only their own attendance history, present count, absent count, recorded count and attendance percentage.

FR13 [P] — A Teacher shall be able to view class-wide reports with student name, register number, present count, absent count, recorded count and percentage.

FR14 [P] — Attendance percentage = present_count / recorded_count × 100. If recorded_count is zero, show “N/A”. Unrecorded dates do not count as absences.

FR15 [P] — Reports shall reflect committed database state. The UI shall not show save success until FastAPI confirms it.

FR16 [P] — When a Teacher creates a student, the system shall generate a proposed username from the student's name and set the initial password to the configured default `student123`. The Teacher shall be able to edit the username and initial password before confirming/assigning the credentials. The final username must be unique.

FR17 [P] — The system shall store only a secure password hash, never the plain-text password. The student account shall be linked to the newly created student record, and the student shall be able to log in with the assigned credentials and view only their own attendance.

FR18 [P] — The Teacher Dashboard shall fetch summary metrics from the backend/database rather than hardcoded sample values. It shall display total students, active students, today's Present and Absent counts, and today's attendance percentage based on recorded statuses. If no attendance has been recorded for the day, the percentage shall display N/A.

4 Business rules and validation

Rule	Required behavior
BR01	Student name required; trim whitespace; 2–100 characters.
BR02	Register number required; 1–30 characters; unique case-insensitively.
BR03	Class/section required; 1–50 characters.
BR04	Attendance date required in YYYY-MM-DD format; future dates are not allowed.
BR05	Attendance status is exactly Present or Absent.
BR06	Every active student must have a status before a teacher saves a full register.
BR07	A student has at most one attendance record per date.
BR08	Students cannot create or edit their attendance, even by direct API requests.
BR09	A Student may view only the student record linked to their authenticated account.
BR10	Inactive students are excluded from new registers by default; history is retained.
BR11	Bulk attendance writes commit atomically; failed saves leave no partial records.
BR12	Attendance percentages use only persisted records; zero recorded records yields N/A.
BR13	When creating a student account, propose a username derived from the student's name; enforce uniqueness and allow the Teacher to edit it before confirming credentials.
BR14	The default initial password is student123 for this educational MVP; the Teacher may change it before assigning credentials. Store only a secure hash, and never log or persist the plaintext password.
BR15	Create the student row and linked Student user account atomically. If either insert fails, roll back both changes.

Rule	Required behavior
BR16	Teacher Dashboard metrics must be fetched from persisted database records. Today's percentage is Present / (Present + Absent) × 100; if no statuses are recorded, show N/A.

Example

A student has 18 Present and 2 Absent records. Their recorded count is 20 and attendance percentage is 18 / 20 × 100 = 90%.

5 Web interface specification

Screen	Required content and behavior
Login	Username/identifier and password; validation; generic invalid-credentials message; route by server-verified role.
Student Dashboard / My Attendance	Own present count, absent count, recorded count, percentage and personal attendance history. No editing controls.
Teacher Dashboard	Database-backed total/active student counts; today's Present/Absent counts and percentage based on recorded statuses; navigation to Students, Mark Attendance and Reports.
Students — Teacher	Searchable list with register number, name, class/section and active status; Add, Edit and Deactivate actions.
Mark Attendance — Teacher	Date selector, roster, Present/Absent controls, save action and saved-state feedback.
Daily Register — Teacher	Selected date, statuses, totals and edit controls for saved records.
Reports — Teacher	Class-wide student summary, attendance counts and percentage; search and sorting.

UI01 — All fields shall have visible labels; validation messages shall be understandable. Preserve form entries after validation/network errors.

UI02 — Show loading states; disable duplicate submissions while a request is pending; report success only after API confirmation.

UI03 — Students shall not see teacher-only navigation or controls. This is a usability measure, not a substitute for API authorization.

UI04 — Status shall be shown as text as well as color. Controls shall support keyboard use, visible focus and associated labels.

UI05 — At 375px and 1440px viewport widths, core flows remain usable without page-level horizontal scrolling.

UI06 — Attendance dates shall be stored as SQL DATE values to avoid timezone shifts.

6 Data model and validation

Table / field	Type and rule
users.id	Primary key.
users.username	Required, unique login identifier.
users.password_hash	Required hash; never store plaintext password.
users.role	Required enum/check: Student or Teacher.
users.student_id	Nullable FK to students.id; required for Student role and null for Teacher role.
users.is_active	Boolean; defaults true.
students.id	Primary key.
students.name	Required text, 2–100 characters.
students.register_number	Required text, 1–30 characters; unique case-insensitively.
students.class_name	Required text, 1–50 characters.
students.is_active	Boolean; defaults true.
attendance.id	Primary key.
attendance.student_id	Required FK to students.id.
attendance.attendance_date	Required SQL DATE; no future date.
attendance.status	Required Present/Absent value.
attendance.created_at / updated_at	Server-generated timestamps.
Constraint	Unique(student_id, attendance_date); foreign keys prevent orphan records.

DB01 — PostgreSQL shall enforce unique usernames, unique register numbers, valid roles/statuses, foreign keys and unique attendance per student/date. Index attendance_date and student_id for common queries.

DB02 — The authenticated Student account must map to one student record. A Student account without a valid mapping shall not access attendance data.

DB03 — Use migrations or a repeatable schema setup. The API shall use an ORM or parameterized database access.

7 REST API contract

Base path: /api/v1. JSON request/response bodies. All routes except login and health require authentication. Example route names may be adjusted only if the equivalent contract and tests are maintained.

Method and path	Contract and access
POST /auth/login	Public. Body: username, password. Returns authenticated identity and role; never returns password/hash.
POST /auth/logout	Authenticated. Invalidates session or clears server-managed session as appropriate.
GET /auth/me	Authenticated. Returns current user ID, role and linked student ID where applicable.
POST /students	Teacher only. Create student and linked Student account in one transaction. Propose a name-based username and default initial password; allow Teacher edits before confirming. Return the created student and initial credentials once for handoff; never return/store password hashes.
GET /students	Teacher only. List/search students; optional q and active filter.
GET /students/{id}	Teacher only; return student or 404.
PATCH /students/{id}	Teacher only. Update student fields or active status.
GET /attendance/me	Student only. Return own attendance history and summary based on authenticated student mapping.
GET /attendance	Teacher only. Required attendance_date; return daily roster and statuses.
POST /attendance/bulk	Teacher only. Body: attendance_date and records [{student_id,status}]. Full roster required; atomic save.
PATCH /attendance/{id}	Teacher only. Update status for an existing attendance record; preserve uniqueness.
GET /reports/attendance	Teacher only. Class-wide counts and percentages; optional q/sort_by.
GET /health	Public local-development health endpoint; do not expose secrets.
GET /dashboard/summary	Teacher only. Return total_students, active_students, present_today, absent_today, recorded_today and attendance_percentage (null/N/A when recorded_today is zero). Metrics must be derived from database records.

Example bulk request: {"attendance_date":"2026-10-28","records":[{"student_id":"<student-id-1>","status":"Present"},{"student_id":"<student-id-2>","status":"Absent"}]}

API01 — The API shall validate all inputs and reject unknown fields or invalid enums. Student endpoints derive student identity from the authenticated account; clients cannot supply a different student_id to access another student's records.

API02 — Unauthorized requests return 401; authenticated but role-forbidden requests return 403. Nonexistent resources return 404.

API03 — Bulk attendance validates all records before writing. Duplicate student IDs or a missing active student reject the request without partial writes.

8 Errors and reliability

HTTP status	Meaning
400	Malformed or semantically invalid request.
401	Missing, invalid or expired authentication.
403	Authenticated user lacks permission for the action.
404	Resource does not exist or is not available to the user.
409	Duplicate register number or conflicting attendance data.
422	Field or payload validation failed.
500 / 503	Unexpected server error or temporary dependency outage; no internal details exposed.

REL01 — Bulk attendance writes shall use one PostgreSQL transaction; failure of any write rolls back the entire register.

REL02 — A repeated save for an existing date updates existing student/date records rather than creating duplicates.

REL03 — Authentication secrets and password hashes must not be logged. Errors must not expose stack traces, SQL statements or credentials.

REL04 — If a network response is lost, reload the selected date from the API before the user retries to reconcile the persisted state.

9 Quality requirements and acceptance tests

ID	Requirement and verification
NF01 Security	Enforce role permissions server-side on every protected route; hash passwords; validate inputs; use parameterized queries; keep secrets out of source control.
NF02 Maintainability	Separate React UI, FastAPI routes/business logic and database models.
NF03 Performance	With 1,000 students and 100,000 attendance records in a local test environment, ordinary reads should complete within 2 seconds under light load.
NF04 Usability	Core flows usable at 375px and 1440px; keyboard focus, labels and errors are clear.
NF05 Data integrity	Database constraints prevent duplicate register numbers, duplicate student/date attendance and orphan records.
NF06 Reliability	Successful writes survive backend restart; bulk failure leaves no partial records.
NF07 Testing	Automate role authorization, login validation, student CRUD, attendance save, duplicate prevention and report calculations.

Test	Given and action	Expected result
TC01	Log in with valid Student credentials.	Login succeeds; Student role is returned and Student dashboard opens.
TC02	Log in with invalid credentials.	Generic authentication error; no protected data is returned.
TC03	Student requests own attendance.	200; only linked student's records and summary are returned.
TC04	Student attempts to request another student's attendance.	403/404; no other student's data is disclosed.
TC05	Student calls create/edit attendance endpoint.	403; database state unchanged.
TC06	Student calls student-management or class-report endpoint.	403; no data mutation or class-wide disclosure.
TC07	Teacher logs in and creates a valid student.	Success; student and associated login mapping are saved.
TC08	Create duplicate register number with different case.	Conflict/validation error; only one student remains.
TC09	Teacher saves a complete daily register.	All records persist; counts match selected statuses.
TC10	Submit register with missing student or duplicate student IDs.	Validation error; no partial records saved.
TC11	Teacher saves the same date with changed statuses.	Existing records update; no duplicate rows.
TC12	Attempt to mark a future date.	Validation error; no records written.
TC13	Student has 18 Present and 2 Absent records.	Report shows 20 recorded and 90% attendance.
TC14	Student has no attendance records.	Percentage displays N/A without division-by-zero error.
TC15	Cause database failure during bulk save.	Transaction rolls back; no partial register is saved.
TC16	Use app at narrow viewport and keyboard-only.	Primary controls remain usable with labels and visible focus.
TC15	Teacher creates a student and accepts the proposed username/default password.	Student row and linked user are created atomically; username is unique; password hash is stored; credentials are shown to the Teacher for handoff.
TC16	Two students have the same or similar names during account creation.	Username conflict is detected and a unique username is proposed or required before account creation.
TC17	Teacher edits the proposed username and initial password before confirming.	The chosen unique username is used; only the password hash is persisted.
TC18	Student logs in with assigned credentials after account creation.	Login opens the Student dashboard and the account is linked to the correct student record.
TC19	Teacher opens the dashboard after changing student/attendance data.	Dashboard metrics match PostgreSQL; no hardcoded statistics are shown.
TC20	No attendance has been recorded for today.	Present and Absent counts are zero, recorded count is zero, and attendance percentage displays N/A.

Overall release acceptance

All functional requirements and adopted proposed rules shall trace to passing tests. Both end-to-end journeys—Student login to personal attendance view, and Teacher login to student management and attendance recording—shall pass. No unresolved defect may permit cross-student data access, student mutation of attendance, duplicate attendance, partial bulk saves or incorrect percentages. This document specifies tests; it does not claim an implementation has passed.

10 Delivery plan and decisions

Implementation sequence

Task 1 — Set up React, FastAPI, PostgreSQL and schema migrations. Implement users, roles and authentication; verify protected routes and role enforcement first.

Task 2 — Implement teacher-only student CRUD and atomic student-account provisioning, including unique username generation, editable initial credentials and student-to-account mapping.

Task 3 — Implement teacher-only daily attendance retrieval, bulk save, edit operations, transactions and duplicate prevention.

Task 4 — Implement Student My Attendance, Teacher reports and a database-backed Teacher Dashboard; verify percentage calculations, summary metrics and personal data isolation.

Task 5 — Execute all acceptance tests, fix gaps, document local setup and review the implementation against this specification.

Decisions requiring review

Decision	Proposed baseline	Effect if changed
Authentication	Username/password with hashed passwords; session/token approach chosen during plan.	Changes login, storage and frontend/API security details.
Student account creation	Teacher creates the student and linked login. Username is proposed from the name and must be unique; initial password defaults to student123 but can be edited by the Teacher before credentials are assigned. Passwords are hashed.	Name collisions require username adjustment. Predictable defaults are acceptable only for this local educational MVP; production should use unique temporary passwords and a forced password change.
Attendance statuses	Present and Absent only.	Additional statuses change validation and report calculations.
Student deletion	Deactivate; preserve history.	Hard deletion changes data-retention and referential-integrity rules.
Deployment	Local educational MVP; production deployment requires additional hardening.	Production use requires HTTPS, secret management, backups and operational controls.

Version 1.2 change summary

This revision clarifies automatic Student account provisioning when a Teacher adds a student, editable initial credentials, unique usernames, secure password hashing, atomic creation of student and user records, and database-backed Teacher Dashboard metrics. Existing scope and role boundaries remain unchanged.

Specification change control

Store this specification with the implementation. For a requirement change, update the requirement, affected UI/API/data contract and tests before changing code. Record version, date, rationale and affected IDs. AI-generated code shall be reviewed against this baseline; a prompt does not override the specification.

Definition of ready

Version 1.2 is the updated baseline for the two-role MVP. It clarifies Teacher-provisioned student accounts and database-backed Teacher Dashboard metrics. The project remains an educational/local MVP and must not be exposed publicly without production-grade access control and deployment hardening.